

天津大学
智能与计算学部
网络空间安全学院

《软件安全》实验报告
实验三：Shellcode 编写实验

姓名：安孟喆
学号：3024244059
年级：2024 级

一. 实验环境

Windows XP 操作系统, Microsoft Visual C++ 6.0 (Win32 Debug) 开发与调试环境。

二. 实验目的

基于给定的示例程序:

1. 分析代码并理解存在的缓冲区溢出漏洞
2. 编写 shellcode 利用发现的缓冲区溢出漏洞实现一个弹出对话框的功能

三. 实验步骤

1. 漏洞程序分析与实验原理

实验程序从 reg.txt 文件读取字符串, 并将其传入 verify 函数。verify 函数内部定义了长度为 44 字节的局部数组 buffer, 但使用 strcpy(buffer, code) 复制输入。strcpy 不会检查目标缓冲区大小, 只在遇到字符串结束符 0x00 时停止, 因此当输入长度超过 44 字节时, 后续数据会继续写入栈中更高地址, 覆盖局部变量、旧 EBP 和函数返回地址。

```
int verify(char *code)
{
    int flag;
    char buffer[44];

    flag = strcmp(REGCODE, code);
    strcpy(buffer, code);
    return flag;
}
```

函数调用时, 栈中保存了局部变量、旧 EBP 和返回地址。函数执行 ret 指令时, CPU 会从栈中取出返回地址并写入 EIP。只要利用越界写入覆盖该返回地址, 就可以使程序从指定地址继续执行。本实验把 Shellcode 放入 buffer, 并把返回地址修改为 buffer 的地址, 使 verify 返回后跳转到栈中的 Shellcode。

2. 创建并正常运行漏洞程序

在 VC6 中创建 Win32 Console Application 空工程 exp, 新建 vuln.cpp, 输入漏洞程序代码。程序启动时先调用 LoadLibraryA("user32.dll"), 以便后续调用 MessageBoxA; 然后从工程根目录中的 reg.txt 读取注册码。

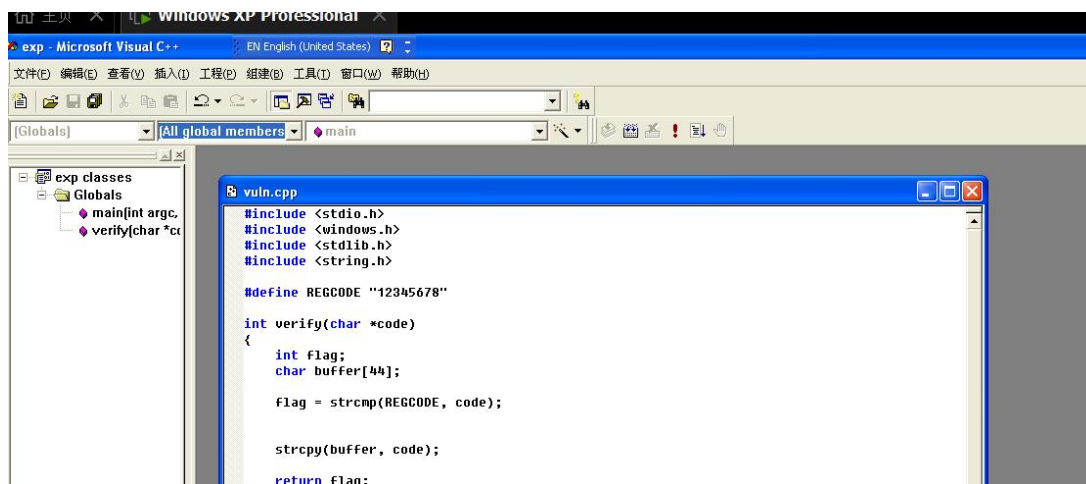


图 1 在 VC6 中创建并编辑漏洞程序

第一次运行时，程序提示 Cannot open reg.txt。原因是 fopen 使用相对路径，VC6 调试运行时的工作目录为工程根目录，而 reg.txt 最初放在 Debug 目录中。

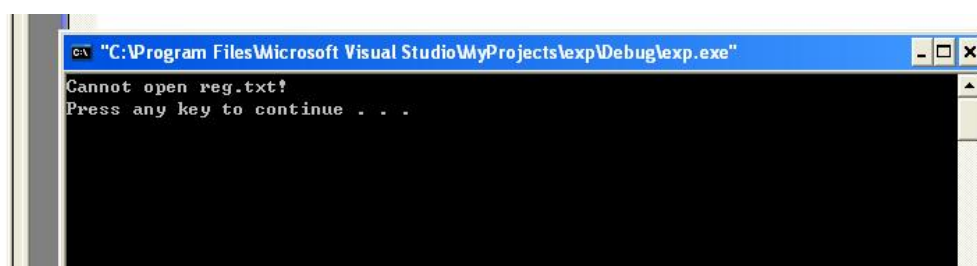


图 2 初次运行时无法打开 reg.txt

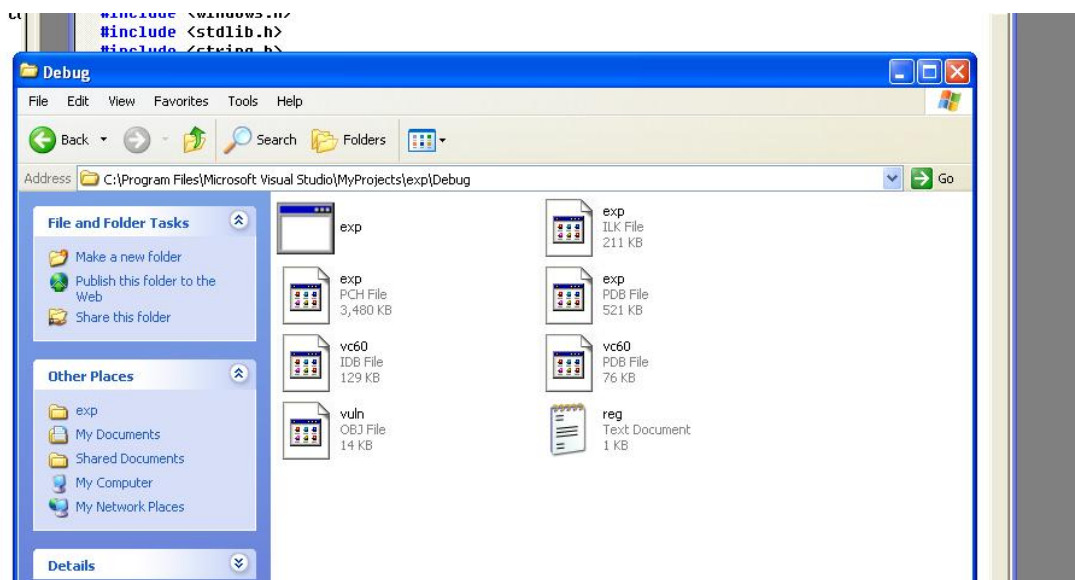


图 3 检查 Debug 目录中的程序文件与 reg 文件

将 reg.txt 放到 exp 工程根目录，并写入正确注册码 12345678 后，程序输出 passed，证明程序能够正常编译、读取文件并完成注册码比较。

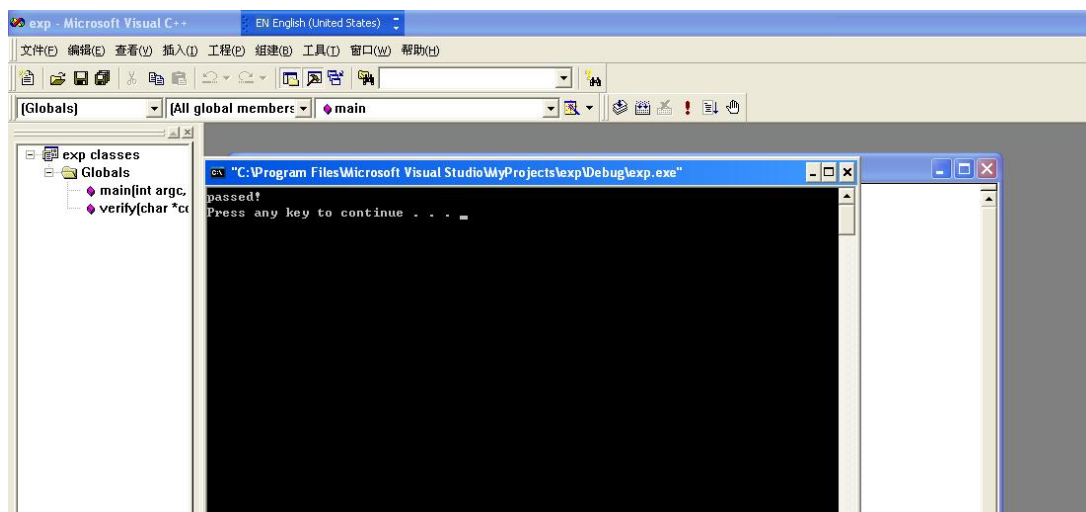


图 4 正确注册码通过程序验证

3. 获取 MessageBoxA 函数入口地址

Shellcode 要弹出对话框，需要调用 user32.dll 中的 MessageBoxA。由于不同系统版本中的函数地址可能不同，本实验新建 getaddr 工程，通过 LoadLibraryA 加载 user32.dll，再使用 GetProcAddress 查询 MessageBoxA 的实际入口地址。

```
HMODULE hUser32 = LoadLibraryA("user32.dll");
FARPROC pMessageBoxA =
    GetProcAddress(hUser32, "MessageBoxA");
```

运行结果显示 user32.dll 的加载基址为 0x7E410000，MessageBoxA 的入口地址为 0x7E4507EA。x86 采用小端序，因此该地址写入机器码时排列为 EA07457E。

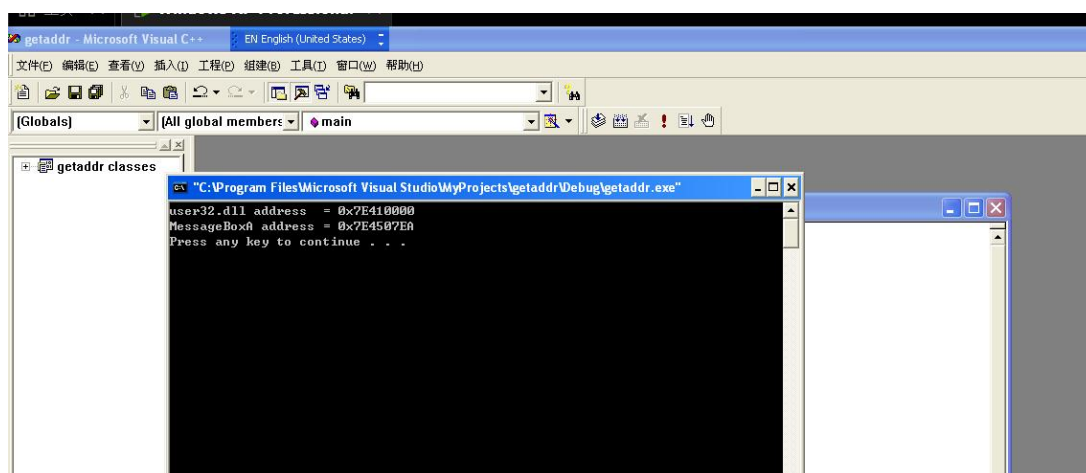


图 5 获取 user32.dll 基址和 MessageBoxA 地址

4. 获取 buffer 地址并观察未初始化内存

回到 exp 工程，在 strcpy(buffer,code) 所在行设置断点并按 F5 启动调试。在 Watch 窗口输入 &buffer[0]，得到 buffer 起始地址为 0x0012FAF0。

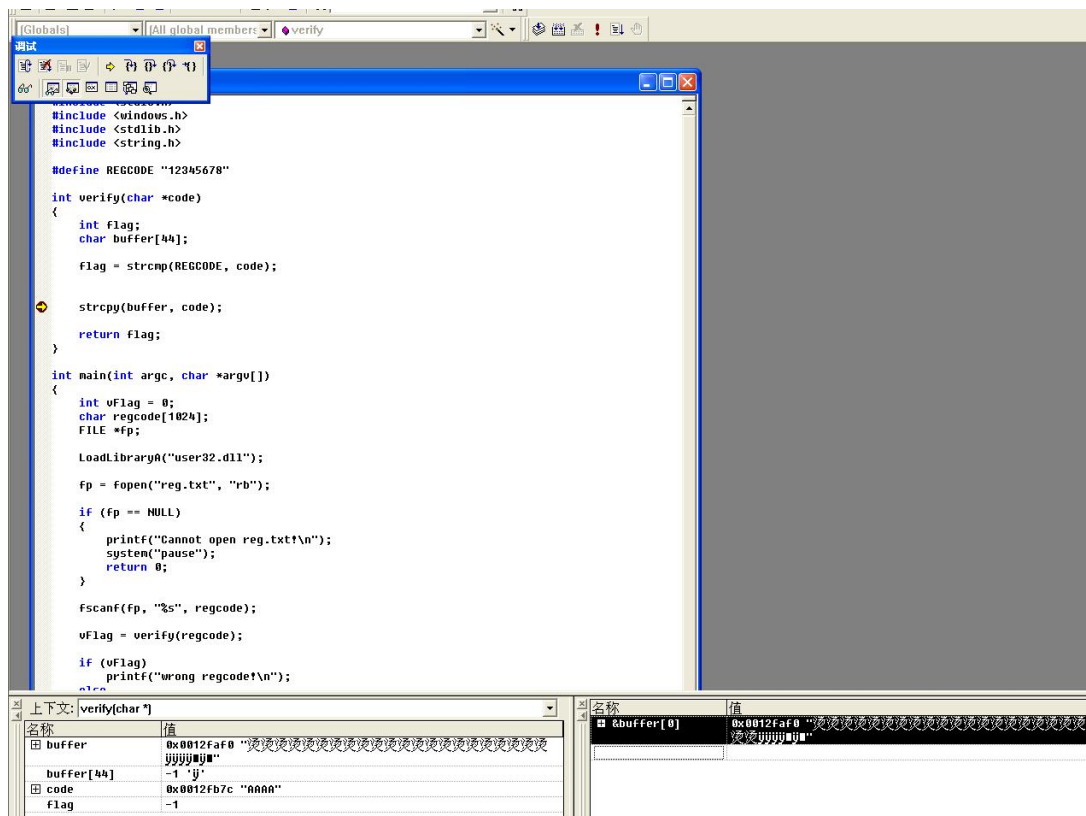


图 6 通过 Watch 窗口获取 buffer 起始地址

断点停在 `strcpy` 执行前时，`buffer` 尚未初始化。VC6 的 Debug 模式通常用 `0xCC` 填充未初始化的栈空间，而连续字节 `CC CC` 在中文代码页中显示为“烫”，因此 Watch 窗口出现大量“烫烫”是正常调试现象，并不表示代码或系统发生故障。

5. 获取 EBP 并计算返回地址偏移

保持程序停在 `verify` 函数内部，打开 `Registers` 窗口，得到 `EBP=0x0012FB20`。函数返回地址保存在 `EBP+4` 处，因此返回地址位置为 `0x0012FB24`。

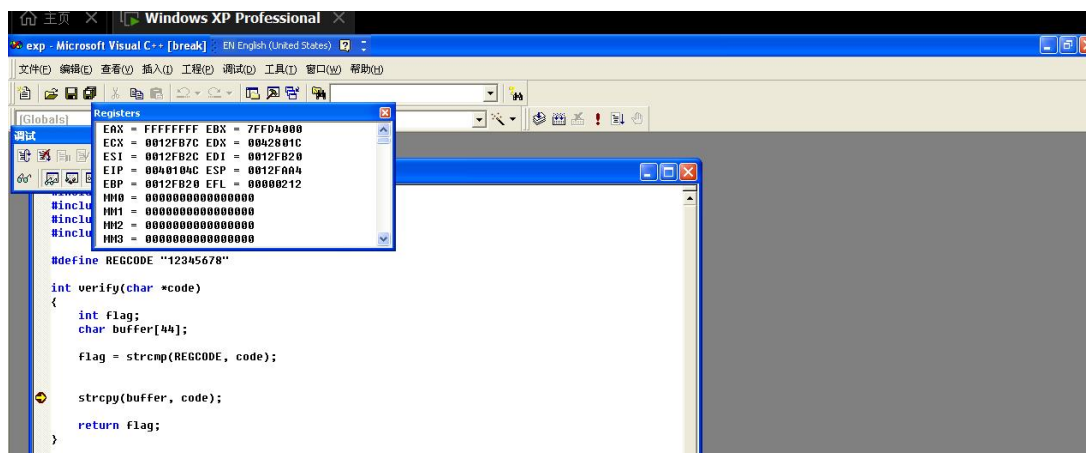


图 7 在寄存器窗口中读取 EBP

```

buffer 起始地址 = 0x0012FAF0
EBP          = 0x0012FB20
返回地址位置 = EBP + 4 = 0x0012FB24

OFFSET = 0x0012FB24 - 0x0012FAF0
        = 0x34
        = 52 字节
  
```

由此可知，输入数据的前 52 字节会覆盖到返回地址之前，第 53~56 字节将覆盖函数返回地址。该结果与 buffer 44 字节、flag 4 字节和旧 EBP 4 字节的典型栈布局相符。

6. 使用特征字节验证 EIP 可控

为了确认偏移计算正确，先生成“52 个 A+4 个 B”的 reg.txt。字符 A 的十六进制为 0x41，字符 B 的十六进制为 0x42。程序直接运行时因栈被破坏而异常退出。

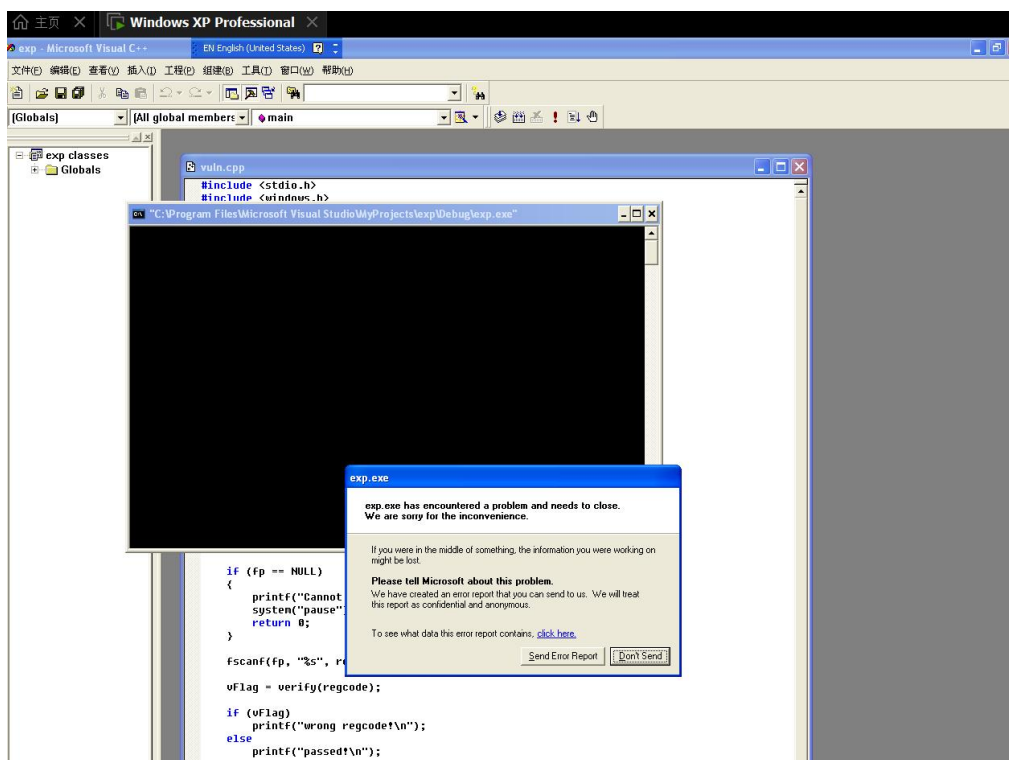


图 8 长输入导致程序异常退出

改用 F5 调试运行后，VC6 捕获到 0xC0000005 Access Violation。

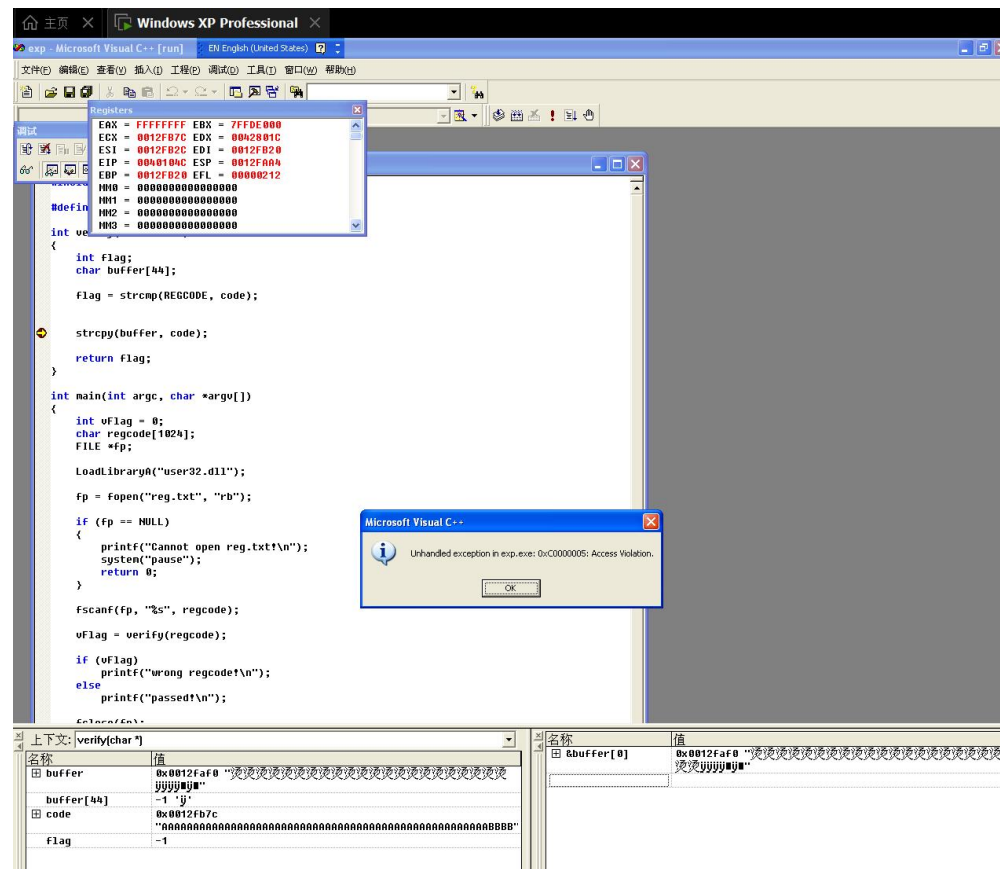


图 9 VC6 捕获访问冲突

进入 break 状态后, Registers 窗口显示 EIP=42424242, 说明返回地址确实被 BBBB 覆盖。此时 0x42424242 不是合法代码地址, 所以反汇编窗口显示“???”并产生访问冲突。这一步证明 52 字节偏移准确, 且程序控制流已经可以由输入控制。

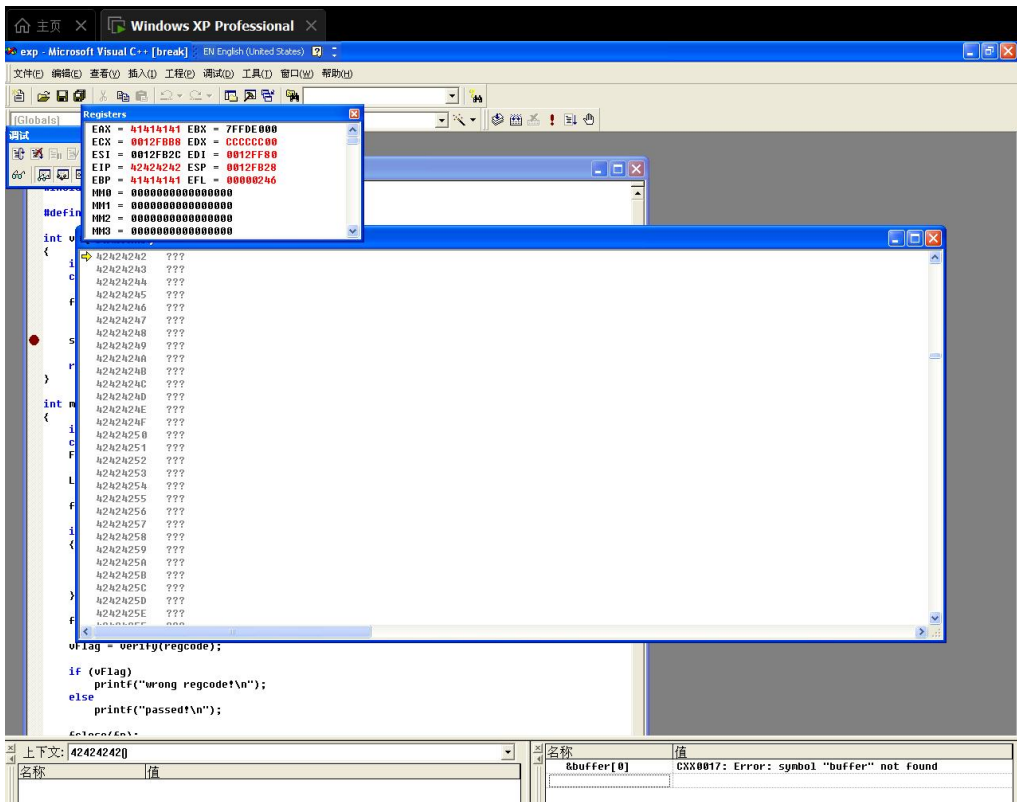


图 10 EIP 被特征字节覆盖为 0x42424242

7. 编写 MessageBoxA Shellcode

实验 Shellcode 的目标是执行 MessageBoxA(NULL, "westwest", "westwest", 0)。首先通过 xor ebx, ebx 将 EBX 清零, 用作 NULL 参数和字符串结束符; 随后将两个“west”压入栈中, 组成字符串“westwest”; 把 ESP 保存到 EAX 后, EAX 即为字符串首地址。根据 Win32 stdcall 调用约定, 四个参数按照从右向左的顺序压栈, 最后调用 MessageBoxA。

33 DB	xor ebx, ebx
53	push ebx
68 77 65 73 74	push 0x74736577
68 77 65 73 74	push 0x74736577
8B C4	mov eax, esp
53	push ebx

50	push eax
50	push eax
53	push ebx
B8 EA 07 45 7E	mov eax, 0x7E4507EA
FF D0	call eax

上述 Shellcode 共 26 字节。由于 buffer 地址为 0x0012FAF0，按小端序写入时应为 F0 FA 12 00。0x00 会使字符串读取提前结束，因此文件中只写入前三个非零字节 F0 FA 12，最后一个 00 由字符串读取后的终止符补齐。

8. 构造最终 Payload

新建 make 工程，用二进制方式生成最终 reg.txt。Payload 的前 26 字节为 Shellcode，从偏移 26 到 51 填入 0x90。0x90 是 NOP 指令，可作为滑板指令；偏移 52 处写入 buffer 地址的低三个字节，使 verify 返回时跳转到 0x0012FAF0。

偏移 0~25:	MessageBoxA Shellcode (26 字节)
偏移 26~51:	NOP 填充 (0x90)
偏移 52~54:	F0 FA 12
末尾终止符:	00
最终返回地址:	0x0012FAF0

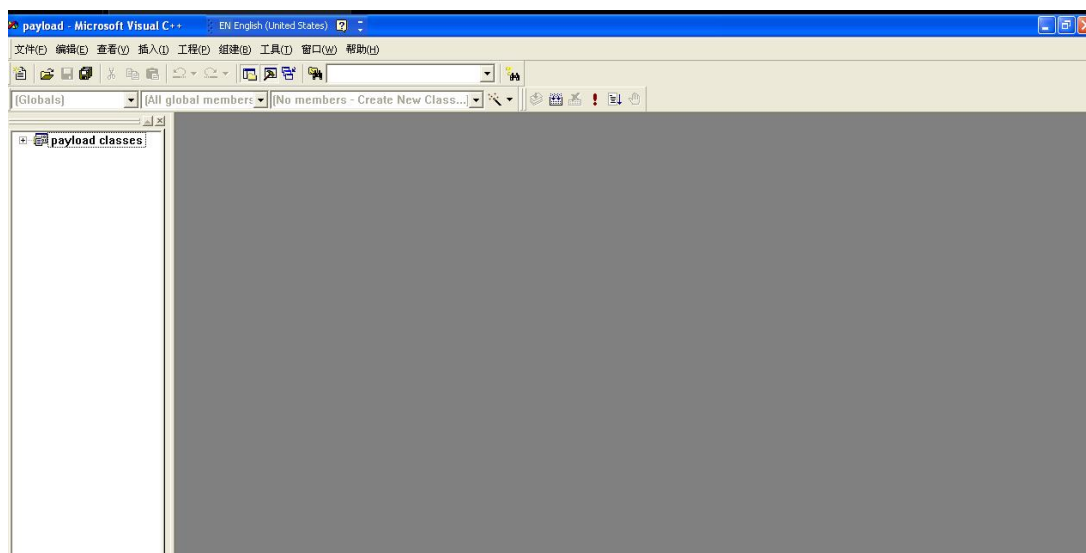


图 11 新建 Payload 生成工程

运行生成程序后，程序输出 Shellcode length=26、Return offset=52、Buffer address=0x0012FAF0 和 MessageBoxA=0x7E4507EA，说明最终 Payload 已按调试得到的参数写入 reg.txt。

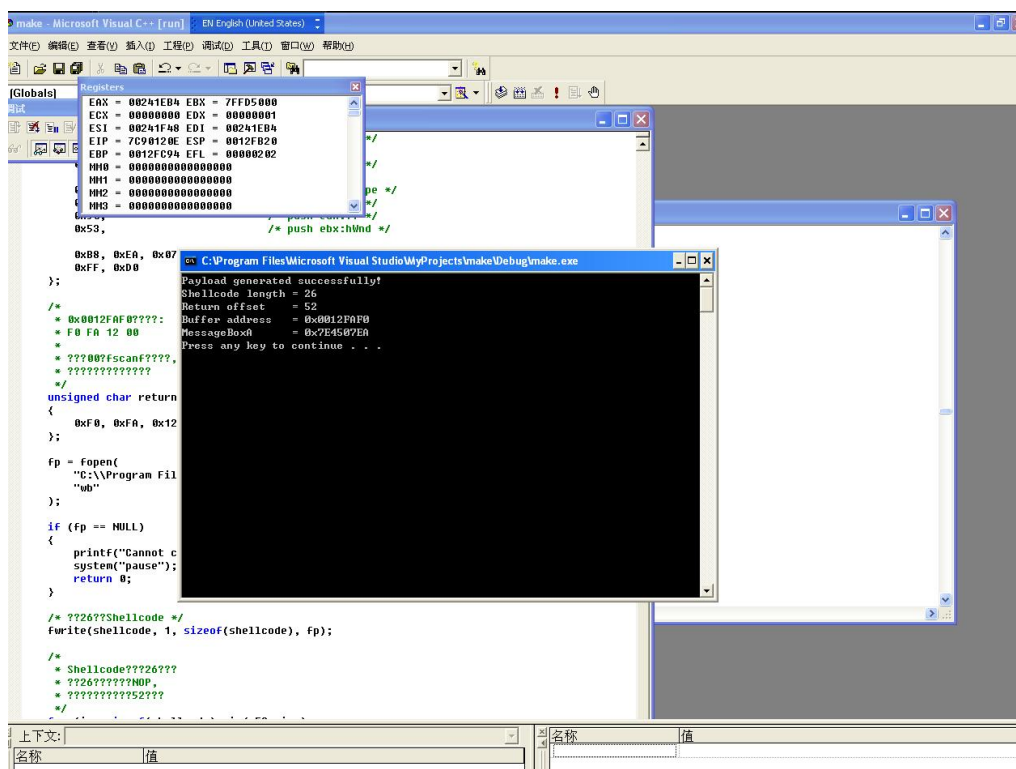


图 12 最终 Payload 生成成功

9. 执行 Shellcode 并验证实验结果

重新打开 exp 工程并使用 F5 调试运行。strcpy 将 Payload 复制到 buffer 后，覆盖原返回地址。verify 执行 ret 时，EIP 被设置为 0x0012FAF0，CPU 开始执行 buffer 中的 Shellcode。Shellcode 将“westwest”压栈并调用位于 0x7E4507EA 的 MessageBoxA，最终成功弹出标题和正文均为 westwest 的对话框。

并检测越界写入。因此，本实验主要用于理解经典漏洞机理，实际软件开发中应检查输入长度、避免使用无边界字符串函数，并启用编译器和操作系统安全防护。

四. 心得体会

通过本次实验，我从程序正常运行开始，逐步完成了漏洞定位、栈帧分析、地址计算、返回地址覆盖验证以及 Shellcode 构造，对栈缓冲区溢出从“程序崩溃”发展到“控制流劫持”的过程有了更加直观的认识。实验中最关键的步骤是分别获得 buffer 地址和 EBP，再利用 $EBP+4-\text{buffer}$ 计算返回地址偏移。使用 52 个 A 和 4 个 B 进行验证后，EIP 显示为 0x42424242，使偏移是否准确一目了然。

在 Shellcode 编写过程中，我进一步理解了小端序、函数调用约定、栈传参和坏字符等概念。例如 MessageBoxA 地址 0x7E4507EA 在机器码中需要写成 EA07457E，而 buffer 地址中的 0x00 不能直接出现在字符串中，需要利用字符串终止符补齐。VC6 调试时出现的“烫烫”也让我认识到 0xCC 是未初始化栈内存的调试填充值。

最终 westwest 对话框成功弹出，证明返回地址已经跳转到 buffer，并执行了植入的 Shellcode。实验也让我认识到，Windows XP 中能够直接完成的利用在现代系统上会受到 ASLR、DEP 和 /GS 等机制限制。因此，在开发程序时应尽量避免 strcpy 等不安全函数，严格检查外部输入长度，并开启编译器安全选项，从源头降低漏洞风险。